Dipartimento di Elettronica e Informazione

Politecnico di Milano
prof. ***Carlo Ghezzi***
prof. ***Angelo Morzenti***
prof. ***Pierluigi San Pietro***

20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (39) 02-2399.3400
Fax (39) 02-2399.3411

Appello di Ingegneria del Software

9 Febbraio 2005

Cognome

Nome

Matricola

Sezione (segnarne una) Ghezzi ☐, Morzenti ☐, SanPietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 1h30m.
6. Punteggio totale a disposizione: 26/30.

Valutazione:
Esercizio 1: ……..

Esercizio 2: ……..

Esercizio 3: ……..

Esercizio 1 (punti 10)

Il metodo statico suGiuOgiuSu riceve come parametro un array *a* contenente tre numeri interi e restituisce al chiamante un valore di tipo *boolean*; se a[0]>a[1] allora se a[1]<a[2] suGiuOgiuSu restituisce true, altrimenti restituisce false; se a[0]<a[1] allora se a[1]>a[2] restituisce true, altrimenti false.

a) Considerando il paragrafo precedente come una specifica del metodo suGiuOgiuSu *evidenziare un suo difetto* particolarmente rilevante, motivando adeguatamente la risposta.

R. La specifica è incompleta, perché non è definito il valore restituito nel caso in cui a[0] è uguale ad a[1].

b) Sempre considerando il primo paragrafo come una specifica del metodo, *fornire 4 casi di test funzionali* per tale metodo, scegliendoli, e motivando tale scelta, sulla base del criterio delle combinazioni proposizionali.

R. Si possono scegliere 4 casi che rendano vere o false le due proposizioni a[0]>a[1] e a[1]>a[2]. Per esempio, si possono quindi ottenere

a[0]	a[1]	a[2]	Val. Restituito
1	2	1	true
1	2	3	false
2	1	3	true
2	1	0	false

c) Si consideri la seguente implementazione del metodo, non necessariamente corretta rispetto alla specifica.

```
static boolean suGiuOgiuSu (int [] a) {
        if ( a[0]>a[1] && a[1]<a[2] )
                return true;
        else    if ( a[0]<a[1] )
                        if ( a[1]>a[2] )
                                return true;
                        else return false;
                else return false;
}
```

Indicare, motivando la risposta, *quanti dati di test occorrono* per effettuare un test strutturale che copra tutte le diramazioni. Fornire un esempio di tali dati di test.

R. Bastano 4 dati di test, che corrispondono ai 4 cammini che portano alle 4 istruzioni return.
1) a[0]>a[1] e a[1]<a[2] e.g. a[0]=2, a[1]=1, a[2]=3
2) a[0]<a[1] e a[1]>a[2] e.g. a[0]=2, a[1]=3, a[2]=1
3) a[0]<a[1] e a[1]<=a[2] e.g. a[0]=2, a[1]=3, a[2]=4
4) a[0]>=a[1] e a[1]>=a[2] e.g. a[0]=3, a[1]=2, a[2]=1

d) Per ottenere la copertura rispetto al criterio delle condizioni *è necessario aggiungere altri dati di test oltre a quelli indicati* al punto (c)? Se sì, indicare quali, se no, spiegare perché.

R. No, perché quelli sopra riportati permettono di rendere vere e false ognuna delle tre condizioni delle istruzioni if e anche ognuna delle due componenti della prima di esse (a[0]>a[1] e a[1]<a[2]).

e) Con riferimento al punto (c), *definire un'eccezione TuttiUgualiException* di tipo checked, che possieda solo due costruttori, uno senza argomenti e uno con un argomento di tipo stringa. *Modificare il codice del metodo suGiuOgiuSu* in modo che esso lanci un'eccezione *TuttiUgualiException* nel caso in cui i tre elementi dell'array siano tutti uguali.

R.

```
class TuttiUgualiException extends Exception {
        public TuttiUgualiException () { super(); }
        public TuttiUgualiException (String s) { super(s); }
}

static boolean suGiuOgiuSu (int [] a) throws TuttiUgualiException {
        if (a[0]==a[1] && a[1]==a[2])
                throw new TuttiUgualiException ("suGiuOgiuSu ");
        if ( a[0]>a[1] && a[1]<a[2] )
                return true;
        else    if ( a[0]<a[1] )
                        if ( a[1]>a[2] )
                                return true;
                        else return false;
                else return false;
}
```

Esercizio 2 (punti 8)

Si vuole definire la classe *ExtremelyLongInt* che permetta di rappresentare e manipolare numeri interi anche estremamente elevati. Allo scopo di facilitare l'effettuazione di calcoli algebrici (p.es., il calcolo del massimo comune divisore e del minimo comune multiplo tra due oggetti *ExtremelyLongInt*) si sceglie di rappresentare un qualsiasi valore *ExtremelyLongInt* con un REP che consista nella scomposizione del numero in fattori primi.

```
class ExtremelyLongInt {
    …
// il REP
    private int [] base;  // contiene tutti i fattori primi dell'oggetto ExtremelyLongInt
    private int [] exp;  // contiene gli esponenti nello stesso ordine dei fattori
    private int segno; //contiene +1 se il numero è positivo, -1 altrimenti
…
}
```

Per esempio, il valore astratto 360 di tipo *ExtremelyLongInt* ha una rappresentazione in cui segno =1. base = {2, 3, 5} ed exp = {3, 2, 1}, dal momento che $360 = 2^3 \cdot 3^2 \cdot 5^1$.

a) Scrivere (senza implementare) l'*invariante di rappresentazione* per tale astrazione, usando formule aritmetiche commentate con frasi in linguaggio naturale.

R.
(segno = -1 || segno =1)
&&base.length = exp. length // i due array hanno la stessa lunghezza
// (tante basi quanti esponenti)
&&∀i (0≤i<exp. length → exp[i]>0) // gli esponenti sono tutti positivi
&& ∀i∀j (0≤i,j<exp. length && i!=j → base[i]!=base[j]) // basi tutte diverse
&&∀i (0≤i<base. length → primo(base[i])) // i fattori sono tutti primi.
dove primo(j) vale true sse j è un numero primo.
Formalmente, primo(j) <=> ¬∃ h>1, k>1 : h*k = j

b) Scrivere la *funzione di astrazione.*

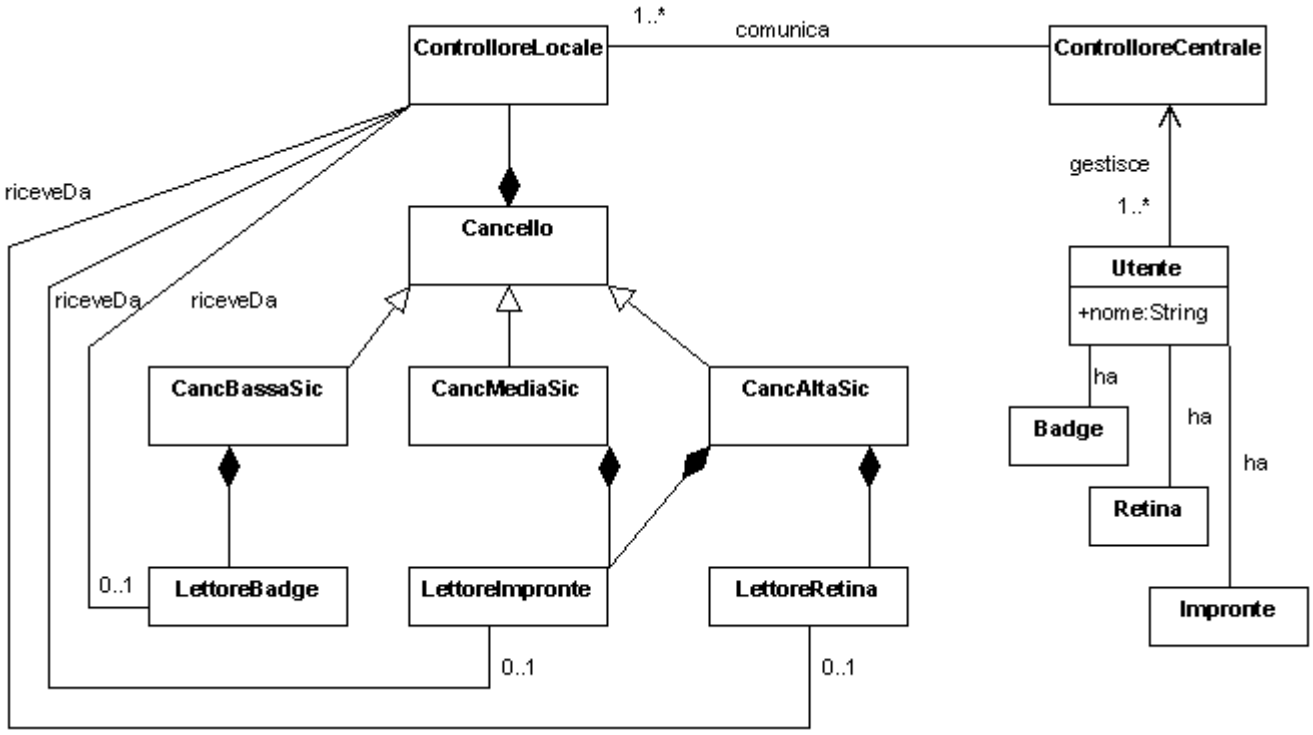
R. La funzione di astrazione prende come argomenti il REP, cioè la coppia di array base e exp, restituisce il valore del numero rappresentato ed è definita come segue

AF(c) = segno* $\prod_{0 \le i < base.\text{length}} base[i]^{\exp[i]}$

Esercizio 3 (punti 8)

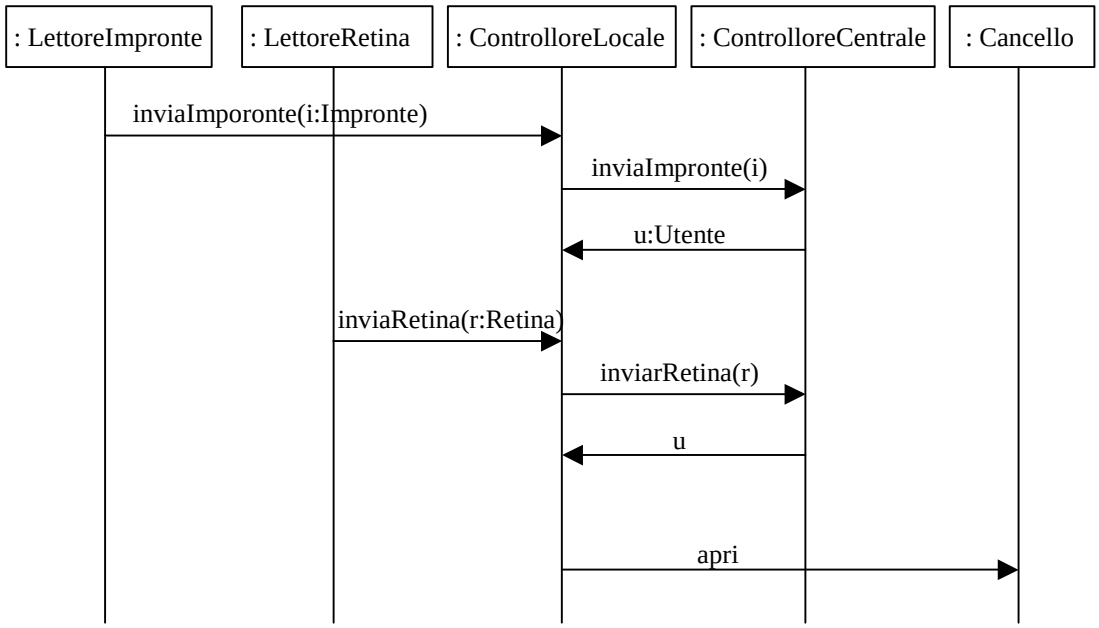
1. Si descriva mediante un class diagram in UML i dati utilizzati dal seguente sistema di controllo degli accessi a un edificio.

Il sistema si compone di un controllore centrale e di una serie di cancelli agli accessi dell'edificio. Il controllore centrale mantiene anche un database con i dati degli utenti che possono accedere all'edificio. Ci sono 3 tipi di cancelli: a bassa, media, ed alta sicurezza. I cancelli a bassa sicurezza verificano l'identità degli utenti solo mediante un lettore di badge. I cancelli a media sicurezza, invece, verificano l'identità mediante un lettore di impronte digitali. Infine, i cancelli ad alta sicurezza verificano l'identità sia mediante un lettore di impronte digitali, che mediante un lettore di retina.
Ogni cancello ha un controllore locale, il quale riceve i dati dai vari lettori e comunica con il controllore centrale per verificare l'identità degli utenti. Ogni utente e' caratterizzato da un nome, da un badge, da delle impronte digitali, e dai dati della sua retina.



2. Si descriva mediante un sequence diagram il seguente caso di funzionamento del sistema.

Un utente arriva ad un cancello ad alta sicurezza, e fa leggere prima le impronte digitali, poi la retina all'apposito lettore. Ogni lettore spedisce i dati al controllore locale, il quale li rimanda al controllore centrale, e ne riceve indietro un oggetto con i dati dell'utente corrispondente. Se l'utente ricevuto dal controllore centrale e' lo stesso entrambe le volte, il controllore locale invia un segnale di apertura al cancello.


: LettoreImpronte
: LettoreRetina
: ControlloreLocale
: ControlloreCentrale
: Cancello
inviaImporonte(i:Impronte)
inviaImpronte(i)
u:Utente
inviaRetina(r:Retina)
inviarRetina(r)
u
apri